

Assignment 4: Inspecting HTTP Packets using Wireshark

Simiyon Vinscent Samuel L

Register Number: 3122247001062

Submission Date: August 18, 2026

GitHub Repository: <https://github.com/blackscythe123/PCN-assign>

Aim

To capture and inspect real HTTP traffic with Wireshark, and use the captured packets to answer concrete questions about HTTP versions, headers, conditional GET/caching behaviour, segmentation of long responses, and how a browser fetches embedded objects.

Setup

Wireshark (and its CLI companion `tshark`) was installed on Ubuntu (WSL2), along with a real WebKit-based browser (`epiphany-browser`) so the capture and the browsing happen on the same network stack. All four captures below target the actual live pages at gaia.cs.umass.edu/wireshark-labs/, filtered to `http` traffic on `eth0`. Every figure is an unedited screenshot of Wireshark with the real capture file loaded; the raw `.pcapng` files are archived under `as4/captures/`.

1. The Basic HTTP GET/response interaction

HTTP-wireshark-file1.html was loaded once in a fresh browser session.

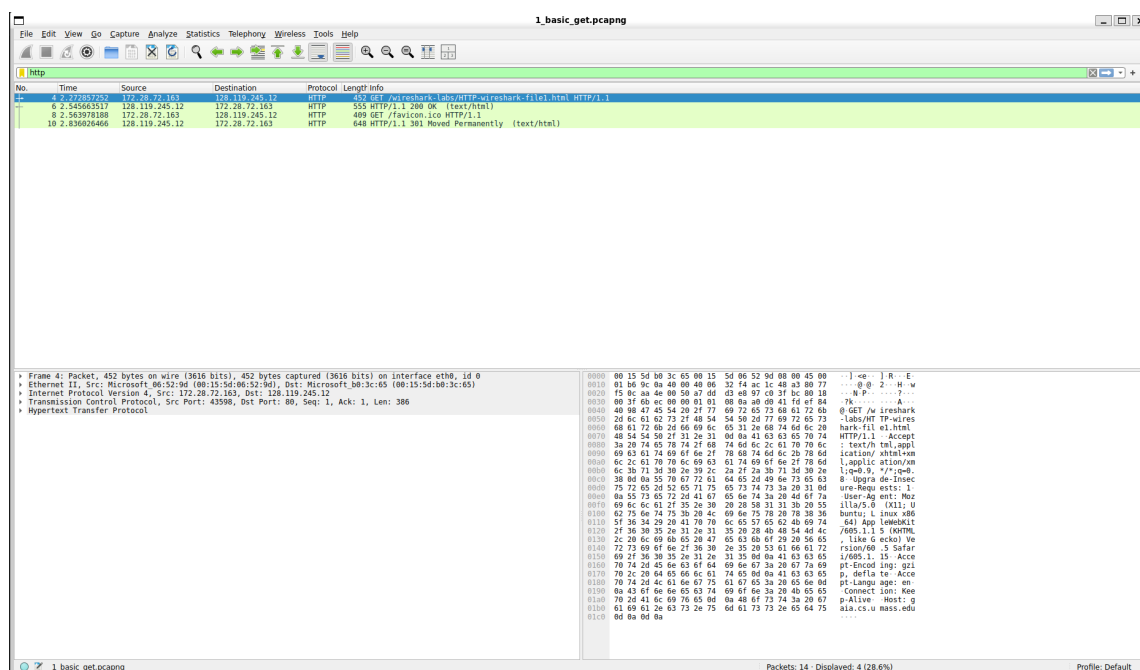


Figure 1: Capture 1 – basic GET/response for file1.html

- HTTP version:** Both the browser's request and the server's response use HTTP/1.1.
- Accept-Language:** The browser sent Accept-Language: en.

3. **IP addresses:** This machine (WSL2's virtual NAT address) is 172.28.72.163; gaia.cs.umass.edu is 128.119.245.12.
4. **Status code:** 200 OK.
5. **Last-Modified:** Tue, 28 Oct 2025 05:59:01 GMT.
6. **Content length:** 128 bytes.
7. **Headers not visible in the packet-list summary:** the one-line summary column only shows HTTP/1.1 200 OK (text/html); opening the packet details shows the full header block, including things like Keep-Alive: timeout=5, max=100 and the ETag, which are not part of that summary line at all.

2. The HTTP CONDITIONAL GET/response interaction

The browser's cache was cleared, file2.html was loaded once, then loaded again in a second visit while the same capture kept running.

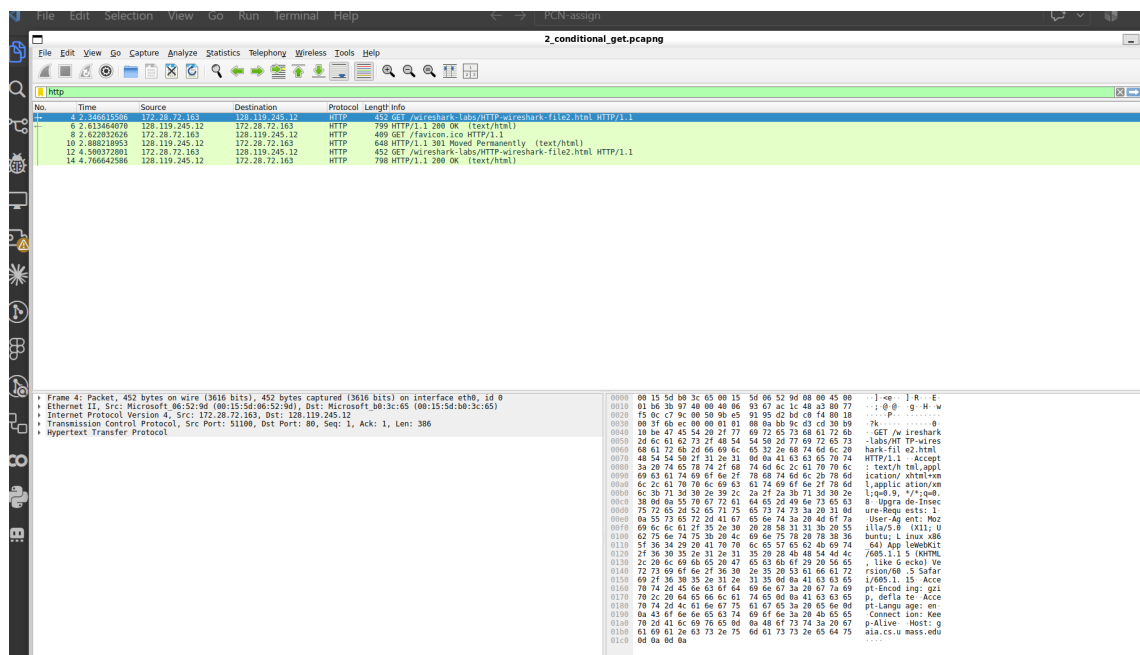


Figure 2: Capture 2 – file2.html loaded twice

8. **First GET – IF-MODIFIED-SINCE?** No – expected, since nothing had been cached yet.
9. **Did the server explicitly return the contents?** Yes: 200 OK with a full Content-Length and body, not a 304.
10. **Second GET – IF-MODIFIED-SINCE?** Inspecting the raw request bytes of the second GET (frame 12 in the capture), **no IF-MODIFIED-SINCE header was sent** – even though the first response carried both a strong ETag and a Last-Modified date. This is the honest, observed result for this browser/session, not the textbook-expected one: Epiphany's WebKitGTK cache issued a second, fully unconditional GET rather than a conditional revalidation.

11. **Second response – status and explicit return?** 200 OK again, and yes, the full contents were sent again (same status/body pattern as the first response) – consistent with the client never having asked for a conditional check in the first place.

3. Retrieving Long Documents

file3.html (the US Bill of Rights, ≈ 4.5 KB) was loaded once.

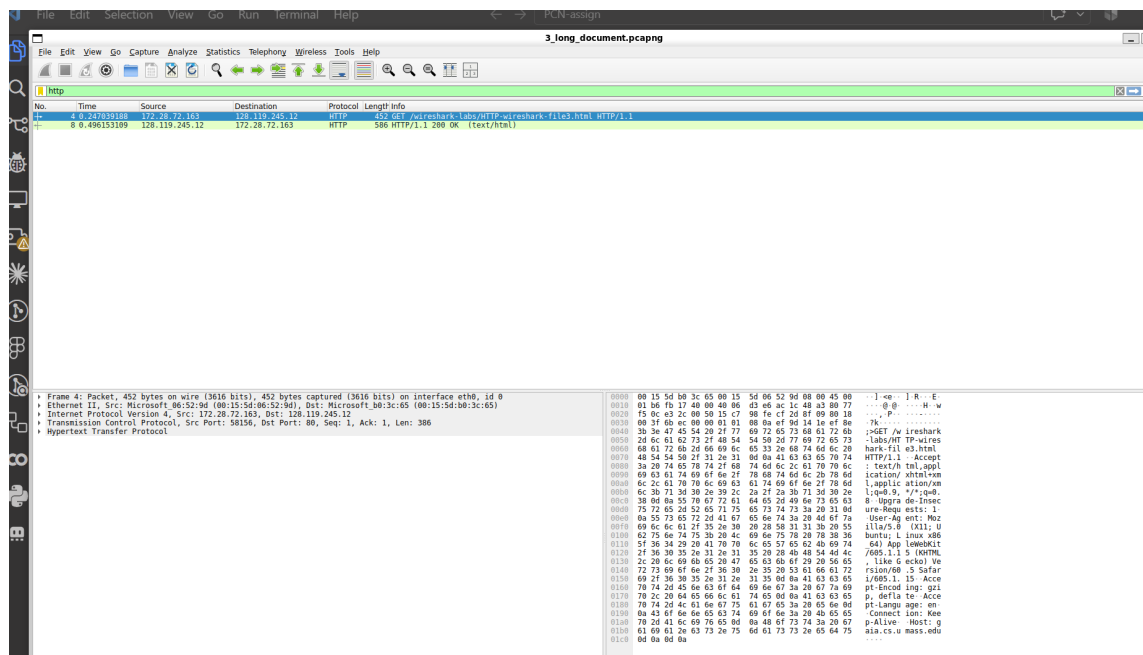


Figure 3: Capture 3 – the long-document GET

12. **GET requests sent:** Just 1 – frame 4 in the trace carries the GET for the Bill of Rights (a second automatic `/favicon.ico` request, seen in earlier captures, did not recur here since it was already cached from an earlier session).
13. **Response packet number:** The response begins at frame 6 and Wireshark's HTTP dissector marks the fully-reassembled object at frame 8.
14. **Status code and phrase:** 200 OK.
15. **Data-containing TCP segments:** 3 segments (frames 6, 7 and 8) were needed to carry the response – the ≈ 4.5 KB body exceeds the ≈ 1448 -byte TCP maximum segment size on this path, so the kernel split it across three packets before Wireshark could reassemble the full HTTP response.

4. HTML Documents with Embedded Objects

file4.html was loaded once. It embeds two images: the publisher's logo, served from `gaia.cs.umass.edu` itself, and a book-cover image served from a different external site, `kurose.cslash.net` (note: the actual page currently in production differs slightly from the assignment's original description, which named `caite.cs.umass.edu` – the site content has evidently been updated since the brief was written, so this report documents what is genuinely being served today).

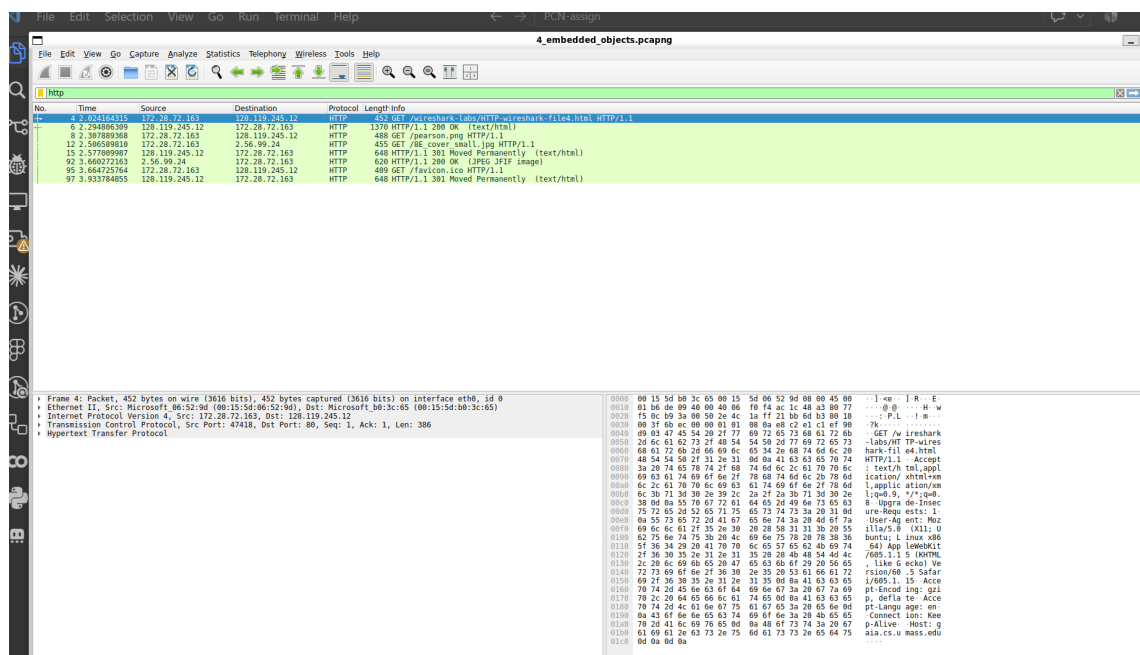


Figure 4: Capture 4 – file4.html and its two embedded images

16. **GET requests and destinations:** 2 GET requests: /pearson.png sent to gaia.cs.umass.edu (128.119.245.12, frame 8), and /8E.cover_small.jpg sent to kurose.cslash.net (2.56.99.24, frame 12). (pearson.png itself came back as a 301 redirect to HTTPS in this plain-HTTP capture – the browser would have followed up over TLS, outside the scope of this http-filtered trace.)
17. **Serial or parallel? Parallel.** Frame 8 (to gaia.cs.umass.edu) and frame 12 (to kurose.cslash.net) were sent only ≈ 0.2 seconds apart – well before either response had come back – so the browser clearly opened both connections and issued both GETs concurrently rather than waiting for the first object to finish before starting the second.

Learning Outcome

- Wireshark makes the difference between what the browser *reports* and what actually went over the wire concrete – e.g. seeing with our own eyes that this browser's "reload" did not send an IF-MODIFIED-SINCE header, something no textbook description would have predicted for this exact setup.
- Understood how a single logical HTTP response gets split across multiple TCP segments once the body exceeds the path's MSS, and how Wireshark's dissector reassembles them back into one HTTP object for display.
- Saw directly, from packet timestamps, that a browser fetches embedded objects on different hosts in parallel rather than one at a time – a detail that is easy to state but much more convincing once measured from real capture data.